

### **1. What is the Collection framework in Java?**

**Ans:** Collection is a set of interfaces and classes in Java that is used to store, delete, retrieve and manipulate data/information that is stored in the form of objects. It has inbuilt methods and classes that can be used according to the requirements to manipulate large amount of data.

### **2. What is the difference between ArrayList and LinkedList?**

**Ans:**

#### **ArrayList**

- It is a class which implements the List Interface only
- It uses a Dynamic Array Data Structure to store its elements
- Using an ArrayList for inserting an element is not as efficient as compared to LinkedList because each element to the right of the desired index needs to be shifted right.
- ArrayList is used when fetching and storing data needs to be efficient.
- It consumes less memory space as it stores only Objects
- It provides random access - in  $O(1)$  time

#### **LinkedList**

- It is a class which implements List and Dequeue Interfaces
- It uses a Doubly Linked List Data Structure to store its elements
- Using a LinkedList for inserting an element at the start, at the end or at any other memory location is more efficient as compared to ArrayList because for every element to be added, a new memory location is created and its address is linked to its neighbours' nodes
- LinkedList provides better manipulation of Data as it has many methods implemented as compared to ArrayList
- It consumes more memory space as it stores Objects as well as their neighbours' memory addresses
- It does not provide random access -  $O(n)$  time

### **3. What is the difference between Iterator and ListIterator?**

**Ans:**

#### **Iterator**

- We can traverse only in forward direction of elements using an Iterator
- It can be used in List, Set and Queue interfaces.
- While traversing a Collection, it can only perform a remove operation and not an add operation

#### **ListIterator**

- We can traverse both forward and backward direction of elements using a ListIterator
- It can be used only in List interface
- While traversing a Collection, it can perform add, remove and set operations

#### **4. What is the difference between Iterator and Enumeration?**

**Ans:**

##### **Iterator**

- An Iterator provides a way to traverse or iterate through the elements of a collection, regardless of whether that collection is a legacy collection (like Vector or Hashtable) or a newer, non-legacy collection (like ArrayList or HashSet).
- Iterator is fail-fast meaning that it throws ConcurrentModificationException if the collection gets/is modified while the iterator is still in use.
- It is slower compared to Enumeration because it performs fail-fast checks and it has a remove() method to safely remove elements from the collection during iteration.
- It is able to perform a remove operation while it traverses through the collection.

##### **Enumeration**

- Unlike Iterator, Enumeration only traverses through legacy collection like ArrayList or HashSet.
- It is not fail-fast meaning that it does not check whether collection is modified or not during a traversal.
- Enumeration is faster as compared to Iterator as it does not perform fail-fast checks nor does it have a more processing method like remove() in Iterator.
- It cannot perform a remove operation while it traverses through the collection.

##### **Enumeration**

#### **5. What is the difference between List and Set?**

**Ans:**

##### **List**

- A List can contain duplicate elements.
- It preserves the Insertion Order of the elements hence making it an Ordered Collection.
- Vector class is the single legacy class which implements List Interface.
- List allows a number of null values as elements.

##### **Set**

- A Set contains only unique elements.
- It is an Unordered Collection as it does not preserve the Insertion Order of elements.
- No legacy classes are implemented by Set Interface.
- Set allows only a single null value as its element.

#### **6. What is the difference between HashSet and TreeSet?**

**Ans:**

##### **HashSet**

- It provides an unordered collection of elements.
- It allows duplicate elements.
- HashSet is implemented using a HashTable.
- It has a Time Complexity of  $O(1)$  for adding, removing and checking the occurrence of an element.
- It uses less memory than TreeSet as it only stores the elements.
- Elements in a HashSet are iterated in a random order.

- HashSet is generally preferred when ordering is not important, and fast access and occurrence tests are needed.

### **TreeSet**

- It provides an ordered collection of elements based on their natural ordering.
- It does not allow duplicate elements.
- TreeSet is implemented using a self balancing Binary Search Tree(aka Red-Black Tree).
- It has a Time Complexity of  $O(\log n)$  for adding, removing and checking the occurrence of an element.
- It uses more memory as it stores additional information for maintaining the order rather than just storing the elements.
- Elements in a TreeSet are iterated in sorted order.
- TreeSet is generally preferred when elements need to be sorted or accessed in a specific order.

## **7. What is the difference between Array and ArrayList?**

**Ans:**

### **Array**

- It is a data structure where elements are stored and accessed in the sequential manner.
- It stores only Homogeneous elements in it.
- Array can store primitive data types as well as objects.
- Size of an Array is fixed.
- Elements in an Array are mutable very easily, meaning that it can be changed once it is created.
- Arrays are compatible with traditional for loops and can be easily passed as arguments to other methods.
- Arrays have a limited set of methods for their data manipulation as compared to ArrayList.
- It can be easily initialized at the time of creation.
- Since they are implemented as a contiguous set of memory location holding elements, the performance of Arrays is better such as accessing elements by index.

### **ArrayList**

- It is a class which implements the List interface.
- It stores both Homogeneous and Heterogeneous elements in it.
- ArrayList stores only objects.
- Size of an ArrayList is dynamic and it can be changed later.
- ArrayList is also mutable but the ways to change its elements are by adding, removing or modifying.
- ArrayList requires the use of for-each loop and it may require a more complex code to pass them as arguments to other methods/.
- ArrayList provides a wide range of methods for manipulating its elements like adding, removing and inbuilt sorting methods.
- It requires methods to add elements.
- Since ArrayList uses dynamic memory allocation, it is more memory hence slower performance for certain operations.

